

Student Management System - Program Design Report

Program Design

The program follows a class based design approach. This was a deliberate choice because a class acts as a template from which student objects can be created repeatedly, making it easy to reuse code and represent each student as a self contained unit. The Student class holds both the data for individual students and the logic for managing them, including adding, searching, updating, deleting, and storing records. A class-level attribute `students_list` acts as the student registry, shared across all instances

Key Functions: The most critical function in the system is `search_student`. It takes a registration number as an argument and returns the matching student object if one exists, otherwise it returns `None`. Its importance comes from the fact that both `update_student` and `delete_student` depend on it internally, so any operation that targets a specific student passes through search first.

Student creation collects all required details at once from the user and passes them directly to the constructor. The constructor creates the student object and immediately appends it to `students_list`, so registration and creation happen in a single step.

The `get_all_students` method returns the full `students_list` class attribute, giving access to every student object currently in the registry.

Exception Handling : Two exceptions are handled in the main input loop. The first is the built-in `ValueError`, which is raised when the user enters something that is not a number. Since the program needs to convert the input to an integer before checking if it falls within the valid range of 1 to 7, this conversion is a direct source of exceptions and must be guarded. The second is a custom `InvalidInputError` exception, defined specifically for cases where the input is a valid number but falls outside the accepted range. This separates two distinct failure modes cleanly rather than collapsing them into one handler.

The finally block runs regardless of whether an exception was raised or not. This makes it the right place for the debug log that records what option the user selected.

Data Storage

Student details are persisted across two files. The fields, name, registration number, and student number, are written to a CSV file using Python's `csv.DictWriter`. The remaining personal details, address, contact, and program, are written to a JSON file using `json.dump`. Both files are written in a single call to `store_student_details`, which iterates over `students_list`.

Testing Results

student_system.log

2026-06-27 12:39:03,355 [INFO] New student added , Name: James, RegNo: 20/U/01

2026-06-27 12:39:50,116 [INFO] New student added , Name: Wassajja, RegNo: 24/U/01

2026-06-27 12:40:18,453 [INFO] Student Name of student 20/U/01 updated to Mbappe

2026-06-27 12:40:21,347 [INFO] Update Successful: Updated details of student with registration number 20/U/01

2026-06-27 12:40:27,772 [INFO] student details stored in csv with extra details stored in jason file

2026-06-27 12:40:46,878 [INFO] Student with registration number 20/U/01 deleted

2026-06-27 12:40:56,490 [WARNING] 8 is not a valid option, Enter a number between 1 and 6

Students.csv

name,registration_no,student_no

Mbappe,20/U/01,2000701

Wassajja,24/U/01,2400701

Students.json

```
{  
  "address": "ntinda",  
  "contact": "0712345678",  
  "program": "BSSE"  
},  
{  
  "address": "mbalwa",  
  "contact": "0787765432",  
  "program": "BMAM"}
```

